Practical 9

Multithreading & Synchronization

Part A — Practice Programs (Hour 1)

1) Counter Race — shared counter without and with synchronization

Code

```
public class CounterRace {
    static int unsafeCount = 0;
    static int safeCount = 0;
    static final Object LOCK = new Object();



    static synchronized void safeIncrement() {
        int c = safeCount;
        safeCount = c + 1;
    }

    static void runThreads(Runnable task) throws InterruptedException {
        Thread[] ts = new Thread[4];
        for (int i = 0; i < ts.length; i++) {
            ts[i] = new Thread(task);
            ts[i].start();
        }
        for (Thread t : ts) t.join();
    }

    public static void main(String[] args) throws InterruptedException {
        int perThread = 1000000;
        System.out.println("4 threads x " + perThread + " increments, expected " + (4 * perThread));
        for (int run = 1; run <= 3; run++) {
            unsafeCount = 0;
            runThreads(() -> {
                for (int i = 0; i < perThread; i++) {
                    int c = unsafeCount;   // read
                    unsafeCount = c + 1;   // write (count++ is these two steps; another thread can
                        slip in between)
                }
            });
            System.out.println("Run " + run + " without synchronization: " + unsafeCount);
        }
        for (int run = 1; run <= 3; run++) {
            safeCount = 0;
            runThreads(() -> { for (int i = 0; i < perThread; i++) safeIncrement(); });
            System.out.println("Run " + run + " with synchronized:       " + safeCount);
        }
    }
}
```

Output

```
4 threads x 1000000 increments, expected 4000000
Run 1 without synchronization: 3712792
Run 2 without synchronization: 4000000
Run 3 without synchronization: 4000000
Run 1 with synchronized:       4000000
Run 2 with synchronized:       4000000
Run 3 with synchronized:       4000000
```

The lost updates are timing dependent: some runs print a total that is too low (as in run 1 above), other runs get lucky. The synchronized version is exact every time.

2) Seat Booking Race — oversell without synchronization

Code

```
import java.util.concurrent.atomic.AtomicInteger;

public class SeatBooking {
    static int seatsLeft;
    static final AtomicInteger success = new AtomicInteger();

    static void bookUnsafe() {
        if (seatsLeft > 0) {
            try { Thread.sleep(10); } catch (InterruptedException e) { }
            seatsLeft--;
            success.incrementAndGet();
        }
    }

    static synchronized void bookSafe() {
        if (seatsLeft > 0) {
            try { Thread.sleep(10); } catch (InterruptedException e) { }
            seatsLeft--;
            success.incrementAndGet();
        }
    }

    static void run(Runnable task) throws InterruptedException {
        seatsLeft = 5;
        success.set(0);
        Thread[] ts = new Thread[10];
        for (int i = 0; i < ts.length; i++) {
            ts[i] = new Thread(task);
            ts[i].start();
        }
        for (Thread t : ts) t.join();
    }

    public static void main(String[] args) throws InterruptedException {
        run(SeatBooking::bookUnsafe);
        System.out.println("Without synchronization: " + success.get() + " bookings succeeded,
            seatsLeft = " + seatsLeft
                + (success.get() > 5 ? "  -> OVERSOLD" : ""));
        run(SeatBooking::bookSafe);
        System.out.println("With synchronized book(): " + success.get() + " bookings succeeded,
            seatsLeft = " + seatsLeft);
    }
}
```

Output

```
Without synchronization: 10 bookings succeeded, seatsLeft = -5  -> OVERSOLD
With synchronized book(): 5 bookings succeeded, seatsLeft = 0
```

3) Array Sum (For Advanced Learners — home) — wrong total, then two fixes

Code

```
import java.util.concurrent.atomic.AtomicLong;

public class ArraySum {
    static long total;
    static final Object LOCK = new Object();

    interface Adder { void add(long[] data, int from, int to); }

    static long run(long[] data, Adder adder) throws InterruptedException {
        int n = 4, chunk = data.length / n;
        Thread[] ts = new Thread[n];
        for (int i = 0; i < n; i++) {
            final int from = i * chunk, to = (i == n - 1) ? data.length : from + chunk;
            ts[i] = new Thread(() -> adder.add(data, from, to));
            ts[i].start();
        }
        for (Thread t : ts) t.join();
        return total;
    }

    public static void main(String[] args) throws InterruptedException {
        long[] data = new long[2_000_000];
        for (int i = 0; i < data.length; i++) data[i] = 1;
        System.out.println("Expected sum: " + data.length);

        total = 0;
        long t0 = System.nanoTime();
        long r1 = run(data, (d, f, t) -> { for (int i = f; i < t; i++) total += d[i]; });
        System.out.printf("No synchronization      : %d (%.1f ms)%n", r1, (System.nanoTime() - t0)
            / 1e6);

        total = 0;
        t0 = System.nanoTime();
        long r2 = run(data, (d, f, t) -> { for (int i = f; i < t; i++) synchronized (LOCK) { total
            += d[i]; } });
        System.out.printf("Fix 1: synchronized add : %d (%.1f ms)%n", r2, (System.nanoTime() - t0)
            / 1e6);

        total = 0;
        t0 = System.nanoTime();
        long r3 = run(data, (d, f, t) -> {
            long local = 0;
            for (int i = f; i < t; i++) local += d[i];
            synchronized (LOCK) { total += local; }
        });
        System.out.printf("Fix 2: local partial sum: %d (%.1f ms)%n", r3, (System.nanoTime() - t0)
            / 1e6);
    }
}
```

Output

```
Expected sum: 2000000
No synchronization      : 1397201 (40.1 ms)
Fix 1: synchronized add : 2000000 (41.4 ms)
Fix 2: local partial sum: 2000000 (13.7 ms)
```

Measured with System.nanoTime(). Fix 2 (each thread adds into a local variable and updates the shared total once) is fastest because it takes the lock only 4 times instead of 2,000,000 times.

Part B — Project Work (Hour 2): MiniBank

Many users use a bank at the same time. This lab runs 10 threads against one Account, shows the race condition, and fixes it with synchronized. Files from Practicals 6 to 8 not listed below are unchanged.

3) Account — deposit() and withdraw() made synchronized (depositUnsafe() kept only for the demo)

Code

model/Account.java

```
package model;

import exception.BankException;
import exception.DailyLimitExceededException;
import exception.InsufficientFundsException;
import exception.InvalidAmountException;
import model.annotation.Id;
import model.annotation.MaxLength;
import model.annotation.Positive;

import java.util.Objects;

public class Account implements Transactable, InterestBearing {
    public static final long DAILY_LIMIT = 10000;

    @Id
    @MaxLength(10)
    protected String accountNumber;
    @Positive
    protected long balance;
    private double rate;
    private long withdrawnToday;

    public Account(String accountNumber, long initialBalance, double rate) {
        this.accountNumber = accountNumber;
        this.balance = initialBalance;
        this.rate = rate;
    }

    // Thread-safe: only one thread at a time may run a synchronized method of this object
    @Override
    public synchronized void deposit(long amount) throws InvalidAmountException {
        if (amount <= 0) {
            throw new InvalidAmountException(amount);
        }
        balance += amount;
    }

    // Original (unsafe) version kept only to demonstrate the race condition
    public void depositUnsafe(long amount) {
        long current = balance;       // read
        Thread.yield();               // let another thread interleave
        balance = current + amount;   // write back (may overwrite another thread's update)
    }

    @Override
    public synchronized boolean withdraw(long amount)
            throws InsufficientFundsException, InvalidAmountException, DailyLimitExceededException {
        if (amount <= 0) {
            throw new InvalidAmountException(amount);
        }
        if (withdrawnToday + amount > DAILY_LIMIT) {
            throw new DailyLimitExceededException(DAILY_LIMIT);
        }
        if (amount > balance) {
            throw new InsufficientFundsException(amount - balance);
        }
        balance -= amount;
        withdrawnToday += amount;
        return true;
    }

    public void transfer(Account to, long amount) throws BankException {
        boolean done = false;
        try {
            withdraw(amount);
            to.deposit(amount);
            done = true;
        } catch (BankException e) {
            System.out.println("Transfer failed: " + e.getMessage());
            throw e;
        } finally {
```

```
            System.out.println("Transfer attempt " + accountNumber + " -> "
                    + to.getAccountNumber() + " finished, success=" + done);
        }
    }

    @Override
    public double interestRate() {
        return rate;
    }

    @Override
    public long getBalance() {
        return balance;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    @Override
    public String toString() {
        return "Account[" + accountNumber + ", balance=" + balance + "]";
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) {
            return true;
        }
        if (!(o instanceof Account)) {
            return false;
        }
        Account other = (Account) o;
        return accountNumber.equals(other.accountNumber);
    }

    @Override
    public int hashCode() {
        return Objects.hash(accountNumber);
    }
}
```

1) AccountWorker — Runnable that deposits many times

Code

service/AccountWorker.java

```
package service;

import exception.InvalidAmountException;
import model.Account;

public class AccountWorker implements Runnable {
    private final Account account;
    private final int times;
    private final long amount;
    private final boolean safe;

    public AccountWorker(Account account, int times, long amount, boolean safe) {
        this.account = account;
        this.times = times;
        this.amount = amount;
        this.safe = safe;
    }

    @Override
    public void run() {
        for (int i = 0; i < times; i++) {
            try {
                if (safe) {
                    account.deposit(amount);       // synchronized
                } else {
                    account.depositUnsafe(amount); // not synchronized
                }
            } catch (InvalidAmountException e) {
                System.out.println(Thread.currentThread().getName() + ": " + e.getMessage());
            }
        }
    }
}
```

2, 4) main — 10 threads, join(), race condition, fix, thread names and states

Code

service/MiniBank.java

```
package service;

import exception.BankException;
import model.Account;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicLong;

public class MiniBank {
    static final int THREADS = 10;
    static final int TIMES = 1000;

    static long runDeposits(boolean safe) throws InterruptedException {
        Account account = new Account("ACC-1", 0, 0.04);
        List<Thread> threads = new ArrayList<>();
        for (int i = 1; i <= THREADS; i++) {
            Thread t = new Thread(new AccountWorker(account, TIMES, 1, safe), "worker-" + i);
            threads.add(t);
        }
        System.out.println("State of worker-1 before start(): " + threads.get(0).getState());
        for (Thread t : threads) {
            t.start();
        }
        System.out.println("State of worker-1 right after start(): " + threads.get(0).getState()
                + " (thread name: " + threads.get(0).getName() + ")");
        for (Thread t : threads) {
            t.join();
        }
        System.out.println("State of worker-1 after join(): " + threads.get(0).getState());
        return account.getBalance();
    }

    public static void main(String[] args) throws Exception {
        long expected = (long) THREADS * TIMES;
        System.out.println("=== Without synchronization (" + THREADS + " threads x " + TIMES + "
            deposits of 1) ===");
        long wrong = runDeposits(false);
        System.out.println("Final balance: " + wrong + " (expected " + expected + ")");

        System.out.println("=== With synchronized deposit() ===");
        long right = runDeposits(true);
        System.out.println("Final balance: " + right + " (expected " + expected + ")");

        System.out.println("=== Supplementary: AtomicLong instead of synchronized ===");
        AtomicLong atomic = new AtomicLong();
        List<Thread> ts = new ArrayList<>();
        for (int i = 0; i < THREADS; i++) {
            Thread t = new Thread(() -> {
                for (int j = 0; j < TIMES; j++) {
                    atomic.incrementAndGet();
                }
            });
            ts.add(t);
            t.start();
        }
        for (Thread t : ts) {
            t.join();
        }
        System.out.println("AtomicLong balance: " + atomic.get());

        System.out.println("=== Supplementary: depositing and withdrawing threads together ===");
        Account acc = new Account("ACC-2", 5000, 0.04);
        List<Thread> mix = new ArrayList<>();
        for (int i = 0; i < 5; i++) {
            mix.add(new Thread(() -> {
                for (int j = 0; j < 500; j++) {
                    try { acc.deposit(2); } catch (BankException e) { }
                }
            }, "depositor-" + i));
            mix.add(new Thread(() -> {
                for (int j = 0; j < 500; j++) {
```

	``
	                    try { acc.withdraw(1); } catch (BankException e) { }
	                }
	            }, "withdrawer-" + i));
	        }
	        for (Thread t : mix) t.start();
	        for (Thread t : mix) t.join();
	        System.out.println("Start 5000 + 5*500*2 - 5*500*1 = 7500, final balance: " +
	            acc.getBalance());
	    }
	}
	``

Output	``` === Without synchronization (10 threads x 1000 deposits of 1) === State of worker-1 before start(): NEW State of worker-1 right after start(): RUNNABLE (thread name: worker-1) State of worker-1 after join(): TERMINATED Final balance: 1002 (expected 10000) === With synchronized deposit() === State of worker-1 before start(): NEW State of worker-1 right after start(): TERMINATED (thread name: worker-1) State of worker-1 after join(): TERMINATED Final balance: 10000 (expected 10000) === Supplementary: AtomicLong instead of synchronized === AtomicLong balance: 10000 === Supplementary: depositing and withdrawing threads together === Start 5000 + 5*500*2 - 5*500*1 = 7500, final balance: 7500 ```

Supplementary problems covered: AtomicLong gives the same correct total, and depositing and withdrawing threads together keep the balance correct.

Package layout, build and tag

Code	```
	minibank/
	 |-- exception/
	 | |-- AccountNotFoundException.java
	 | |-- BankException.java
	 | |-- DailyLimitExceededException.java
	 | |-- InsufficientFundsException.java
	 | +-- InvalidAmountException.java
	 |-- model/
	 | |-- annotation/
	 | | |-- Id.java
	 | | |-- MaxLength.java
	 | | +-- Positive.java
	 | |-- Account.java
	 | |-- InterestBearing.java
	 | |-- Premium.java
	 | |-- PremiumAccount.java
	 | +-- Transactable.java
	 |-- service/
	 | |-- AccountWorker.java
	 | |-- MiniBank.java
	 | +-- WithdrawRule.java
	 +-- util/
	     |-- AnnotationValidator.java
	     +-- Validator.java
	
	javac -d out $(find exception model util service -name "*.java")
	java -cp out service.MiniBank
	cd out
	jar --create --file minibank.jar --main-class=service.MiniBank .
	java -jar minibank.jar
	cd ..
	git add .
	git commit -m "Practical 9 - multithreading and synchronization"
	git tag lab-09
	``

Before the fix the printed balance is less than 10000 and varies from run to run; after synchronized deposit() it is exactly 10000 every run. Two threads can read the same old balance, both add 1 and write back, so one update is lost.

Key Questions / Analysis / Interpretation

1. What is a race condition, and why does it occur here?

A race condition is a bug where the result depends on the timing of threads that share and change the same data. balance += amount is really three steps (read, add, write). If two threads read the same value before either writes, both write the same new value and one deposit is lost, so the final balance is too low.

2. What does the synchronized keyword do?

It gives a thread exclusive use of an object's monitor (lock) while it runs the method or block. Other threads that try to enter a synchronized method on the same object must wait (BLOCKED) until the lock is released. It also makes the changes of one thread visible to the next thread that acquires the lock.

3. What are the states in a thread's life cycle?

NEW (created, not started), RUNNABLE (running or ready to run), BLOCKED (waiting for a monitor lock), WAITING (waiting indefinitely, for example in wait() or join()), TIMED_WAITING (waiting with a time limit, for example sleep(ms)) and TERMINATED (run() has finished). The demo prints NEW before start() and TERMINATED after join().